

TypeScript pour React JS

Sécuriser et structurer vos composants React

Présentation pour les étudiants en Licence 2 Informatique et développeurs Front-End.

LICENCE 2 INFORMATIQUE

DÉVELOPPEMENT FRONT-END

ANNÉE 2025–2026



Pourquoi TypeScript ?

Le problème avec JavaScript

- Pas de typage strict
- Erreurs détectées trop tard
- Bugs difficiles à maintenir

TypeScript = JavaScript + types

Une solution pour des applications plus robustes.



Avantages clés de TypeScript



Détection précoce des erreurs

Identifiez les problèmes avant même l'exécution.



Code lisible et maintenable

Améliore la clarté et la collaboration en équipe.



Autocomplétion intelligente

Accélérez votre développement avec des suggestions précises.



Standard professionnel

Une compétence essentielle sur le marché du travail.

TypeScript dans React : Une synergie parfaite

Fichiers .tsx

Extension spécifique pour le JSX typé.

JSX + Types

Combinez la puissance des deux mondes.

Props typées

Garantissez la validité des données passées aux composants.

Composants fiables

Réduisez les bugs et améliorez la robustesse.



JavaScript vs TypeScript : Un exemple parlant

JavaScript

```
function greet(name) {  
  return "Bonjour " + name;  
}
```

Aucune vérification de type à la compilation.

TypeScript

```
function greet(name: string): string {  
  return "Bonjour " + name;  
}
```

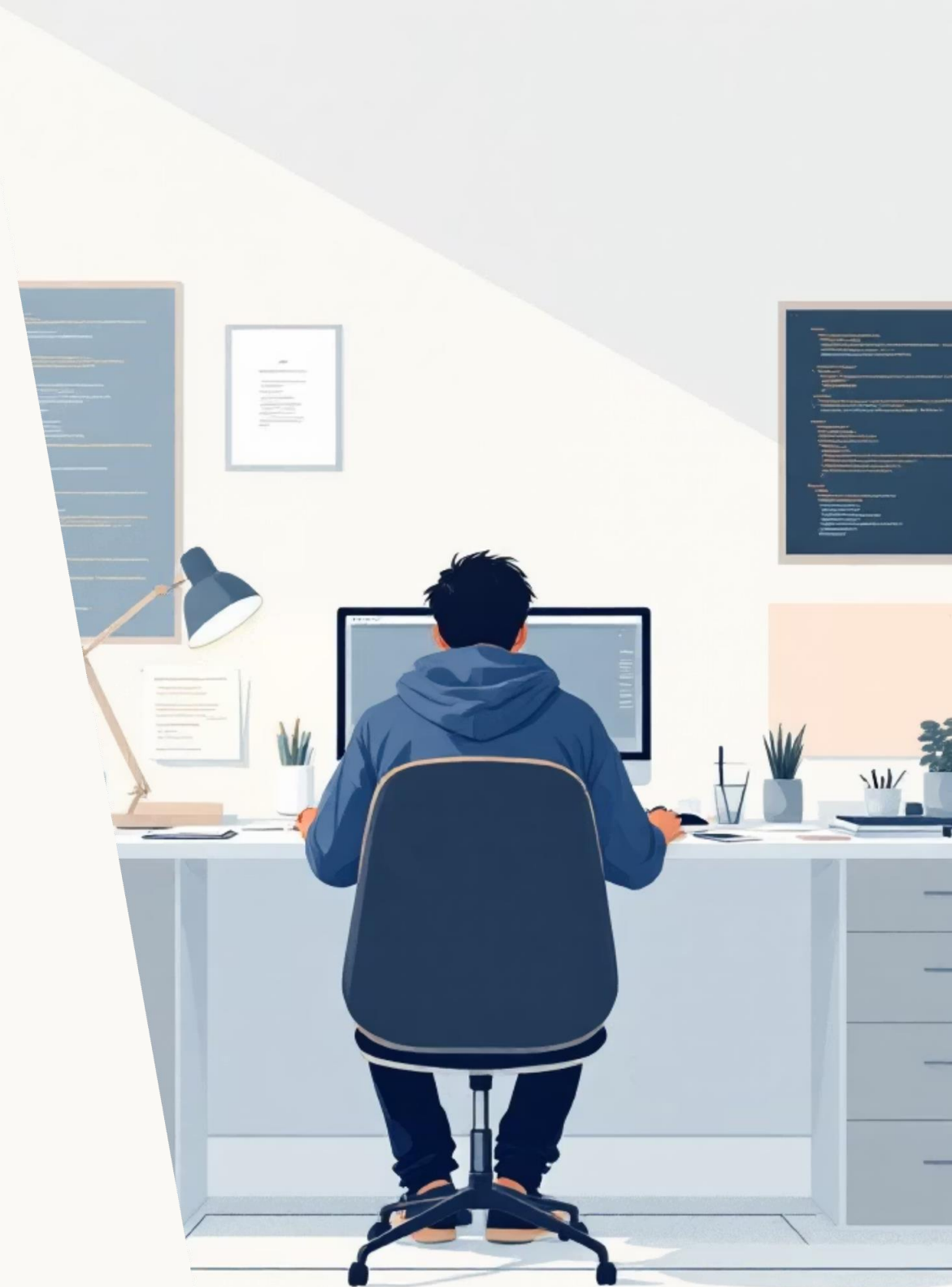
Déclare explicitement les types des paramètres et du retour.

Typage des props : La clé de la sécurité

En définissant les types de vos props, vous assurez que vos composants reçoivent toujours les données attendues, évitant ainsi des erreurs inattendues.

```
type GreetingProps = {  
  name: string;  
};  
  
function Greeting({ name }: GreetingProps) {  
  return <h2>Bonjour {name}</h2>;  
}
```

- ☞ Les props deviennent **sécurisées et prévisibles**, facilitant la compréhension et la maintenance du code.



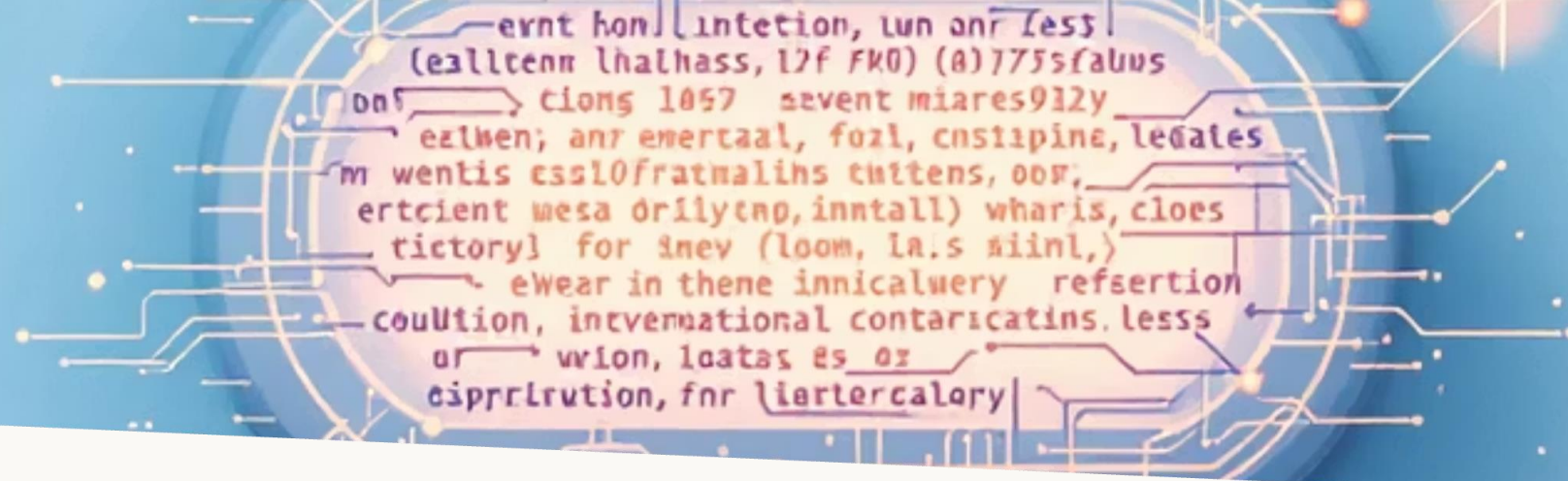
Props multiples et optionnelles : Flexibilité typée



```
type CourseProps = {  
  title: string;  
  year: number;  
  teacher?: string;  
};
```

- Le symbole `?` indique une prop optionnelle.
- Permet une plus grande flexibilité dans l'utilisation des composants.
- TypeScript gère automatiquement l'absence de ces props.

☐ Cette fonctionnalité est cruciale pour des composants réutilisables qui s'adaptent à différents scénarios.



Typage des événements : Interagir en toute sécurité

Le typage des événements est essentiel pour gérer les interactions utilisateur (clics, saisies, etc.) de manière fiable et sans erreur.

```
const handleClick = (  
  event: React.MouseEvent<HTMLButtonElement>  
) => {  
  console.log("Click détecté !");  
  // Accédez aux propriétés de l'événement en toute sécurité,  
  // par exemple: event.currentTarget.value  
};
```

☞ Très utile pour les formulaires et les boutons, garantissant que vous utilisez les bonnes propriétés de l'événement.

Erreurs fréquentes à éviter avec TypeScript



Oublier de typer les props

Annule les bénéfices de la vérification des types.



Utiliser `any` trop souvent

Désactive la sécurité des types pour cette variable.



Confondre `string` / `number`

Des erreurs de type qui peuvent passer inaperçues.



Mélanger `.jsx` et `.tsx`

Crée des incohérences dans la base de code.



TP – TS pour React

[Cliquez ici pour obtenir voir le document](#)

À retenir : L'essentiel de TypeScript pour React

01

TypeScript rend React plus sûr

Réduisez les bugs et augmentez la robustesse de vos applications.

02

Les props doivent être typées

Garantissez une interface claire et prévisible pour vos composants.

03

Moins de bugs, plus de clarté

Améliorez la productivité et la collaboration en équipe.

04

Indispensable en projet réel

Une compétence maîtresse pour tout développeur Front-End.

Maîtriser TypeScript est un atout majeur pour votre carrière en développement web.

